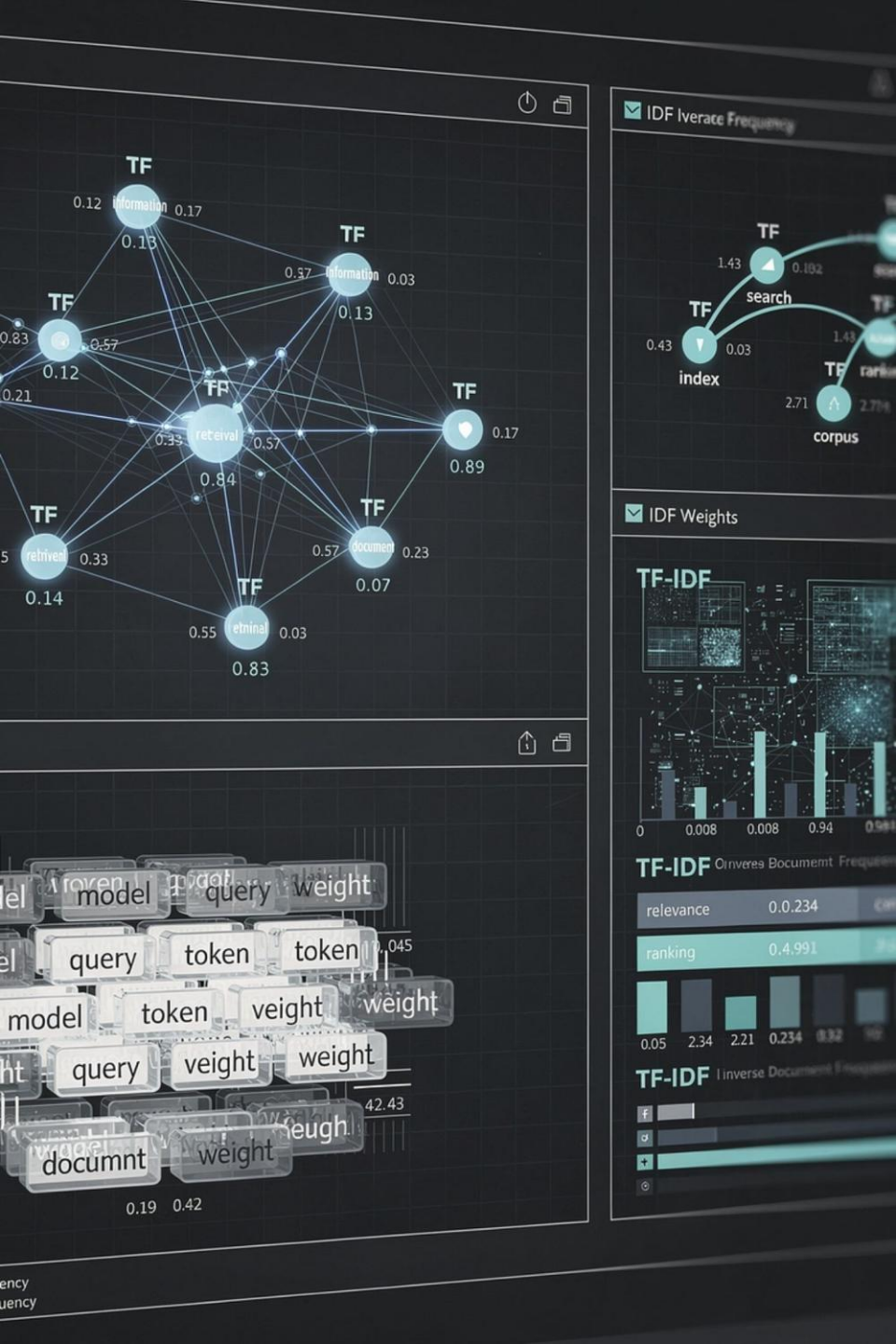




Menerapkan Pembobotan dalam Sistem Temu Kembali Informasi

Term Weighting, TF, IDF, TF-IDF, dan Bag-of-Words

SISTEM TEMU KEMBALI INFORMASI

PERTEMUAN KE-5 | SEMESTER 5

Capaian Pembelajaran

Setelah mengikuti pertemuan ini, mahasiswa diharapkan mampu memahami dan menerapkan konsep pembobotan term dalam sistem temu kembali informasi secara menyeluruh, mulai dari teori hingga implementasi praktis.

1 Memahami Bag-of-Words

Menjelaskan konsep Bag-of-Words sebagai dasar representasi dokumen dan pentingnya term weighting dalam sistem IR.

2 Menghitung Term Frequency (TF)

Menghitung TF dengan berbagai variasi termasuk raw count, boolean, log normalization, dan augmented TF.

3 Menghitung IDF

Menghitung Inverse Document Frequency untuk mengukur kejarangan term di seluruh koleksi dokumen.

4 Menerapkan TF-IDF

Menerapkan formula TF-IDF secara manual maupun menggunakan library Python (scikit-learn).

5 Ranking Dokumen

Menghubungkan nilai pembobotan TF-IDF dengan mekanisme ranking dokumen dalam sistem pencarian.

6 Implementasi & Penerapan Nyata

Mengimplementasikan TF-IDF dalam kode Python dan menyebutkan penerapannya di dunia nyata.

Pokok bahasan

Pertemuan ke-5 ini dirancang secara sistematis dari konsep dasar hingga implementasi dan penerapan nyata. Berikut adalah peta perjalanan pembelajaran kita hari ini.

01	02	03
Mengapa Perlu Pembobotan?	Model Bag-of-Words (BoW)	Term Frequency (TF)
Motivasi dan kebutuhan term weighting dalam IR	Representasi dokumen sebagai vektor frekuensi kata	Rumus, variasi, dan contoh perhitungan
04	05	06
Inverse Document Frequency (IDF)	TF-IDF & Contoh Lengkap	Implementasi Python
Mengukur kejaranan term di koleksi	Kombinasi $TF \times IDF$ dan studi kasus perhitungan manual	Scikit-learn dan implementasi manual
07		
Penerapan & Latihan		
Aplikasi, tugas, dan ringkasan		



BAGIAN 1

Mengapa Perlu Term Weighting?

Masalah: Tidak Semua Kata Sama Pentingnya

Realita dalam Teks

Dalam sebuah dokumen, kata-kata memiliki tingkat kepentingan yang sangat berbeda. Kata umum seperti "dan", "adalah", "yang" muncul ratusan kali tetapi tidak membawa informasi bermakna. Sebaliknya, kata spesifik seperti "mikroprosesor" atau "algoritma" yang jarang muncul justru menjadi pembeda utama antar dokumen.

- Kata yang muncul sering di **satu dokumen** → kemungkinan besar penting bagi dokumen itu
- Kata yang muncul di **hampir semua dokumen** → kurang diskriminatif
- Frekuensi saja tidak cukup untuk menentukan kepentingan sebuah term

Tujuan Term Weighting

Pembobotan (weighting) bertujuan untuk memberikan skor numerik pada setiap term sehingga sistem dapat secara otomatis membedakan dokumen yang relevan dari yang tidak relevan terhadap suatu query.

Dengan pembobotan yang tepat, sistem dapat:

- Mengidentifikasi term kunci yang merepresentasikan topik dokumen
- Mengurangi pengaruh kata-kata umum yang tidak informatif (stopwords implisit)
- Menghasilkan ranking dokumen yang lebih akurat dan relevan
- Meningkatkan nilai Precision dan Recall secara keseluruhan



Pembobotan adalah fondasi dari semua model IR modern, termasuk Vector Space Model, BM25, dan bahkan neural retrieval.

Dampak Tanpa Pembobotan

Jika sistem temu kembali informasi tidak menggunakan pembobotan, berbagai masalah serius akan muncul yang berdampak langsung pada kualitas hasil pencarian.

Semua Kata Dianggap Setara

Tanpa pembobotan, kata "sistem" dan kata "mikroprosesor" dianggap memiliki nilai yang sama. Padahal, kata umum seperti "sistem" hampir tidak membantu membedakan satu dokumen dengan dokumen lainnya, sementara kata spesifik seperti "mikroprosesor" sangat diskriminatif.

Bias terhadap Dokumen Panjang

Dokumen yang lebih panjang secara alami memiliki frekuensi term yang lebih tinggi dibandingkan dokumen pendek. Tanpa normalisasi, dokumen panjang akan selalu mendominasi ranking meskipun dokumen pendek mungkin lebih relevan terhadap query yang diberikan.

Ranking Tidak Akurat

Query seperti "sistem komputer" akan sangat dipengaruhi oleh kata umum "sistem" yang muncul di hampir setiap dokumen teknis. Hasilnya, dokumen yang tidak relevan bisa muncul di peringkat teratas hanya karena mengandung banyak kata umum yang ada di query.

Penurunan Precision & Recall

Akibat ranking yang tidak akurat, nilai Precision (proporsi dokumen relevan di hasil teratas) dan Recall (proporsi dokumen relevan yang berhasil ditemukan) akan menurun drastis, membuat sistem tidak dapat diandalkan untuk pencarian informasi nyata.

BAGIAN 2

Model Bag-of-Words (BoW)

Model Bag-of-Words (BoW)

Apa itu Bag-of-Words?

Bag-of-Words (BoW) adalah model representasi dokumen yang menyederhanakan teks menjadi kumpulan (bag) kata-kata tanpa memperhatikan urutan atau struktur gramatikal. Setiap dokumen direpresentasikan sebagai vektor numerik berdasarkan frekuensi kemunculan kata.

Analogi sederhana: bayangkan Anda mengambil semua kata dari sebuah dokumen, mencampurnya dalam sebuah kantong (bag), lalu menghitung berapa kali setiap kata muncul. Inilah representasi BoW.

- **Urutan kata diabaikan** – "anjing menggigit manusia" dan "manusia menggigit anjing" menghasilkan vektor yang sama
- **Hanya kemunculan yang dihitung** – frekuensi menjadi fitur utama
- **Vocabulary** – semua term unik dari seluruh koleksi membentuk dimensi vektor

Posisi BoW dalam IR

BoW menjadi dasar dari hampir semua model IR klasik karena kesederhanaan dan efektivitasnya. Model-model yang dibangun di atas BoW antara lain:

- **Vector Space Model (VSM)** – representasi dokumen sebagai vektor di ruang multidimensi
- **TF-IDF** – memberikan bobot berbeda pada setiap term dalam vektor BoW
- **BM25 (Okapi BM25)** – variasi probabilistik yang masih digunakan di Elasticsearch
- **Naive Bayes Classifier** – klasifikasi dokumen berbasis probabilitas term



BoW sederhana namun powerful – menjadi titik awal yang solid sebelum memahami model yang lebih kompleks seperti Word2Vec atau BERT.

Ilustrasi Bag-of-Words

Mari kita lihat bagaimana dua dokumen pendek direpresentasikan sebagai vektor BoW. Proses ini membentuk dasar dari semua perhitungan TF-IDF yang akan kita pelajari.

 **Dokumen 1**
"sistem temu kembali informasi penting"

 **Dokumen 2**
"sistem komputer sangat penting"

 **Vocabulary (7 term)**
[sistem, temu, kembali, informasi, penting, komputer, sangat]

Term	sistem	temu	kembali	informasi	penting	komputer	sangat
Dok 1	1	1	1	1	1	0	0
Dok 2	1	0	0	0	1	1	1

Vektor BoW Dok 1: [1, 1, 1, 1, 1, 0, 0] – Vektor BoW Dok 2: [1, 0, 0, 0, 1, 1, 1]. Kedua dokumen berbagi dua term yang sama ("sistem" dan "penting"), namun berbeda pada term-term lainnya, yang mencerminkan perbedaan topik.



BAGIAN 3

Term Frequency (TF)

Term Frequency (TF) – Konsep Dasar

Definisi

Term Frequency (TF) mengukur seberapa sering suatu term t muncul di dalam sebuah dokumen d . Prinsip dasarnya adalah: semakin sering sebuah term muncul dalam dokumen, semakin besar kemungkinan term tersebut relevan dan penting bagi topik dokumen itu.

Rumus dasar TF (Raw Count):

$TF(t, d)$ = jumlah kemunculan term t dalam dokumen d

Misalnya, jika kata "komputer" muncul 5 kali dalam suatu dokumen yang terdiri dari 100 kata, maka $TF(\text{"komputer"}, d) = 5$.

Interpretasi TF

Nilai TF yang tinggi mengindikasikan bahwa term tersebut merupakan topik atau konsep utama dalam dokumen. Namun perlu diingat bahwa TF saja belum cukup – term dengan TF tinggi belum tentu memiliki nilai diskriminatif yang baik jika term tersebut juga sering muncul di dokumen-dokumen lain.

Contoh sederhana:

- Dokumen: *"sistem komputer sistem informasi sistem"*
- $TF(\text{"sistem"}) = 3$
- $TF(\text{"komputer"}) = 1$
- $TF(\text{"informasi"}) = 1$

Dari nilai TF di atas, "sistem" adalah term yang paling dominan dalam dokumen ini. Namun apakah ini berarti "sistem" lebih penting? Itulah mengapa kita membutuhkan IDF!

Variasi Term Frequency

Raw count tidak selalu menjadi pilihan terbaik untuk merepresentasikan TF. Ada beberapa variasi yang sering digunakan tergantung pada kebutuhan dan karakteristik koleksi dokumen.

Variasi TF	Formula	Keterangan	Kapan Digunakan
Raw Count	$TF = f(t, d)$	Jumlah kemunculan mentah	Baseline, dokumen dengan panjang seragam
Boolean TF	1 jika muncul, 0 jika tidak	Hanya memperhatikan ada/tidaknya term	Ketika frekuensi dianggap tidak relevan
Log Normalization	$1 + \log(f(t, d))$	Mengurangi dominasi term yang terlalu sering muncul	Dokumen dengan distribusi term yang tidak merata
Augmented TF	$0.5 + 0.5 \times (f(t,d) / \max f)$	Normalisasi berdasarkan term terfrekuen	Untuk mengurangi bias panjang dokumen
L2 Normalization	$f(t,d) / \sqrt{\sum f^2}$	Normalisasi menggunakan norma Euclidean	Digunakan bersama cosine similarity

 Log Normalization adalah pilihan populer karena mencegah term yang muncul 100 kali dianggap 100x lebih penting dari yang muncul 1 kali – hubungan yang jarang benar dalam praktiknya.

Contoh Perhitungan TF dengan Log Normalization

Mari kita bandingkan hasil Raw TF dan Log Normalization pada dokumen yang sama untuk memahami perbedaan dan keunggulan masing-masing metode.

Dokumen Input

"sistem komputer sistem informasi sistem"

Term	Raw TF	Log TF = $1 + \log(f)$
sistem	3	$1 + \log(3) \approx \mathbf{1.477}$
komputer	1	$1 + \log(1) = \mathbf{1.000}$
informasi	1	$1 + \log(1) = \mathbf{1.000}$

Analisis Perbandingan

Dengan Raw TF, perbedaan antara "sistem" (3) dan "komputer" (1) adalah faktor 3 kali. Ini bisa membuat "sistem" mendominasi terlalu kuat dalam perhitungan akhir.

Dengan Log Normalization, perbedaannya menjadi lebih moderat: 1.477 vs 1.000. Selisihnya hanya ~47%, bukan 200%. Hal ini lebih realistis secara semantik karena sebuah kata yang muncul 3 kali tidak berarti 3x lebih penting.

Catatan penting: Logaritma yang digunakan umumnya adalah log basis 10 atau ln (natural log). Scikit-learn menggunakan ln secara default.

BAGIAN 4

Inverse Document Frequency (IDF)

Inverse Document Frequency (IDF) – Konsep Dasar

Mengapa Perlu IDF?

TF saja tidak mampu membedakan term yang informatif dari term yang umum. Kata "sistem" mungkin muncul di hampir setiap dokumen teknis – TF-nya tinggi, tetapi nilainya sebagai pembeda dokumen sangat rendah. IDF hadir untuk mengoreksi kelemahan ini.

IDF mengukur seberapa **langka/rare** suatu term di seluruh koleksi dokumen:

- Term yang muncul di **sedikit dokumen** → IDF tinggi → lebih diskriminatif
- Term yang muncul di **hampir semua dokumen** → IDF rendah → kurang informatif

Rumus IDF Klasik

Rumus dasar IDF yang paling umum digunakan adalah:

$$\text{IDF}(t) = \log (N / \text{df}(t))$$

- **N** = total jumlah dokumen dalam koleksi
- **df(t)** = jumlah dokumen yang mengandung term t (document frequency)

Semakin kecil df(t) (term jarang muncul di berbagai dokumen), semakin besar nilai IDF(t). Ini adalah mekanisme "penghargaan" untuk term-term yang spesifik dan informatif.

📌 Perhatikan bahwa IDF dihitung berdasarkan seluruh koleksi, bukan per dokumen. Ini adalah komponen "global" yang melengkapi komponen "lokal" TF.

Variasi Formula IDF

Seperti TF, IDF juga memiliki beberapa variasi yang dikembangkan untuk mengatasi kasus-kasus khusus atau meningkatkan performa sistem.

IDF Dasar

$$\log(N / df)$$

Formula klasik dari Sparck Jones (1972). Masalah: jika suatu term muncul di semua dokumen ($df = N$), maka $IDF = 0$. Jika term tidak muncul ($df = 0$), terjadi pembagian dengan nol.

Smooth IDF

$$\log(1 + N / df)$$

Variasi yang menambahkan 1 di dalam logaritma untuk menghindari nilai nol. Digunakan secara default oleh scikit-learn dalam implementasi `TfidfVectorizer`.

IDF dengan Offset +1

$$\log(N / (df + 1)) + 1$$

Menambahkan 1 pada penyebut untuk menghindari pembagian nol, dan menambahkan 1 pada keseluruhan agar term yang ada di semua dokumen tetap memiliki bobot > 0 .

Probabilistic IDF

$$\log((N - df) / df)$$

Berdasarkan pendekatan probabilistik. Memberikan nilai IDF negatif untuk term yang sangat umum ($df > N/2$), yang secara intuitif masuk akal karena term tersebut tidak membantu diskriminasi.

Contoh Perhitungan IDF

Dengan koleksi **N = 1000 dokumen**, mari kita bandingkan nilai IDF dari sebuah term yang sangat umum versus term yang sangat spesifik.

Term	df(t)	Perhitungan	IDF	Interpretasi
sistem	800 dokumen	$\log(1000/800)$	≈ 0.097	Sangat umum – hampir tidak diskriminatif
komputer	200 dokumen	$\log(1000/200)$	≈ 0.699	Cukup umum di bidang teknik
mikroprosesor	5 dokumen	$\log(1000/5)$	≈ 2.301	Sangat spesifik – pembeda yang kuat



Kesimpulan: "mikroprosesor" memiliki IDF ~23x lebih tinggi dari "sistem". Ini berarti dokumen yang mengandung "mikroprosesor" jauh lebih mudah dibedakan dari dokumen lain dibandingkan dokumen yang hanya mengandung "sistem".

BAGIAN 5

TF-IDF: Kombinasi Terbaik

TF-IDF – Menggabungkan Kepentingan Lokal & Global

Formula TF-IDF

TF-IDF adalah produk dari dua komponen yang saling melengkapi:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

- **TF(t, d)** → kepentingan term t di dalam dokumen d secara lokal
- **IDF(t)** → kepentingan global term t di seluruh koleksi

Nilai TF-IDF yang tinggi berarti: term tersebut **sering muncul di dokumen ini** (TF tinggi) DAN **jarang muncul di dokumen lain** (IDF tinggi). Ini adalah definisi ideal dari term yang informatif dan spesifik.

Mengapa Ini Bekerja?

TF-IDF secara elegan mengatasi dua masalah sekaligus:

- Term yang sering muncul di **satu dokumen saja** mendapat TF tinggi + IDF tinggi = skor TF-IDF sangat tinggi (benar-benar penting)
- Term yang sering muncul di **semua dokumen** mendapat IDF mendekati nol = skor TF-IDF sangat rendah (direduksi secara otomatis)
- Term yang jarang muncul di mana pun mendapat TF rendah = skor TF-IDF tetap rendah

Hasilnya adalah pembobotan yang secara otomatis menekan kata umum dan mengangkat kata-kata spesifik yang benar-benar merepresentasikan topik dokumen.

Contoh Perhitungan TF-IDF Lengkap

Dengan N = 1000 dokumen, mari hitung TF-IDF untuk dua term kunci dalam Dokumen A yang mengandung "sistem" sebanyak 4 kali dan "mikroprosesor" sebanyak 2 kali.

Term	TF di Dok A	df(t)	IDF = $\log(N/df)$	TF-IDF = TF × IDF
sistem	4	800	$\log(1000/800) \approx 0.097$	$4 \times 0.097 = \mathbf{0.388}$
mikroprosesor	2	5	$\log(1000/5) \approx 2.301$	$2 \times 2.301 = \mathbf{4.602}$

Interpretasi Hasil

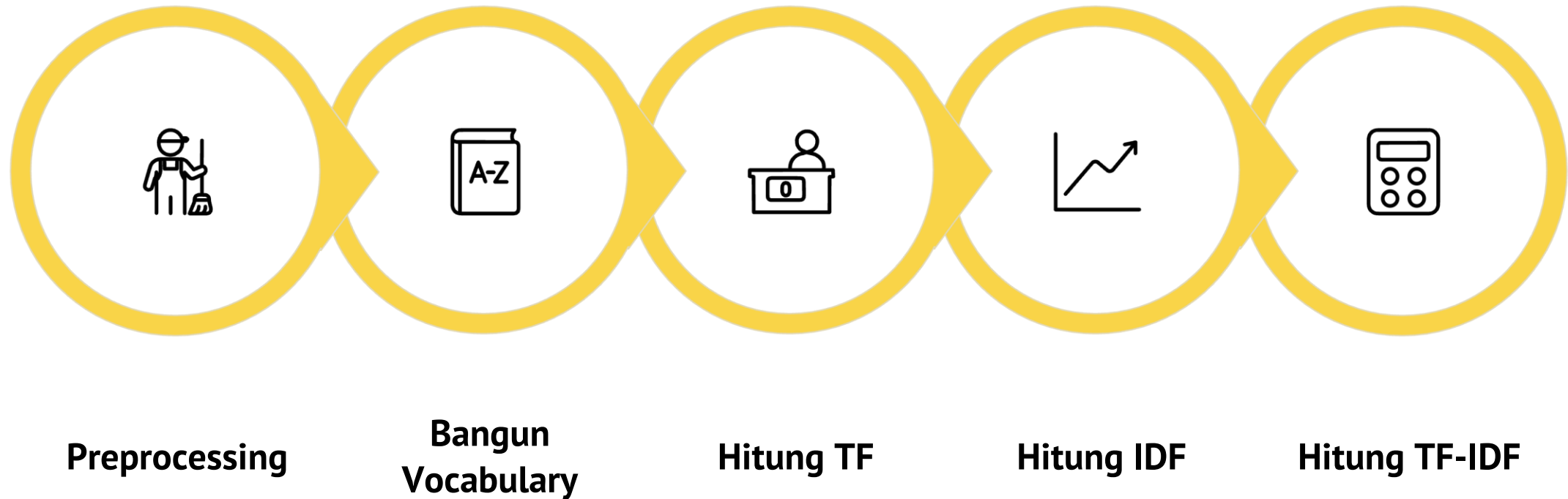
"Mikroprosesor" memiliki TF-IDF hampir **12× lebih tinggi** dari "sistem", meskipun frekuensinya di Dok A lebih sedikit (2 vs 4). Ini membuktikan bahwa kejarangan di koleksi (IDF) jauh lebih berpengaruh dari sekadar frekuensi mentah.

Implikasi untuk Ranking

Jika seorang pengguna mencari "mikroprosesor", Dokumen A akan mendapatkan skor sangat tinggi karena bobot TF-IDF "mikroprosesor"-nya yang besar, menempatkannya di peringkat atas hasil pencarian.

Pipeline: Dari Preprocessing ke TF-IDF

Dalam implementasi nyata, TF-IDF tidak berdiri sendiri. Ia adalah bagian dari pipeline pemrosesan teks yang lengkap. Berikut adalah tahapan lengkap dari teks mentah hingga vektor siap ranking.



Setiap langkah dalam pipeline ini saling bergantung. Kualitas preprocessing sangat memengaruhi kualitas TF-IDF: tokenisasi yang buruk, stemming yang tidak tepat, atau kegagalan menghapus stopwords dapat secara signifikan menurunkan akurasi pembobotan dan ranking dokumen akhir.

Kelebihan & Keterbatasan TF-IDF

✓ Kelebihan TF-IDF

- **Sederhana dan efisien** – mudah diimplementasikan, komputasi cepat bahkan untuk koleksi besar
- **Terbukti efektif** – digunakan di mesin pencari selama puluhan tahun dengan hasil yang solid
- **Interpretable** – nilai bobot mudah diinterpretasikan dan di-debug oleh pengembang
- **Tidak perlu data training** – metode unsupervised, tidak memerlukan label data
- **Adaptif terhadap koleksi** – IDF secara otomatis menyesuaikan dengan karakteristik koleksi dokumen
- **Fondasi yang kuat** – basis untuk BM25, VSM, dan banyak teknik IR lainnya

⚠ Keterbatasan TF-IDF

- **Tidak menangkap semantik** – "kendaraan" dan "mobil" dianggap term yang sama sekali berbeda
- **Mengabaikan urutan kata** – "anjing menggigit manusia" = "manusia menggigit anjing"
- **Sulit menangani sinonim** – dokumen tentang "laptop" dan "notebook" bisa dianggap tidak terkait
- **Polisemi tidak tertangani** – kata "bank" (sungai vs keuangan) tidak dibedakan berdasarkan konteks
- **Sparse representation** – vektor TF-IDF sangat jarang terisi (sparse) untuk koleksi besar
- **Tidak memahami konteks** – berbeda dengan model berbasis neural seperti BERT yang memahami konteks kalimat

❗ Meskipun model modern seperti BERT mengatasi banyak keterbatasan ini, TF-IDF tetap relevan untuk banyak kasus penggunaan karena kecepatan dan kesederhanaannya, terutama sebagai baseline awal dalam proyek IR.

BAGIAN 6

Implementasi dengan Python

Implementasi TF-IDF dengan Scikit-learn

Python dengan library `scikit-learn` menyediakan `TfidfVectorizer` yang siap pakai untuk menghitung TF-IDF secara efisien. Berikut adalah contoh implementasi lengkap beserta penjelasannya.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Dataset: dua dokumen sederhana
dokumen = [
    "sistem temu kembali informasi penting",
    "sistem komputer sangat penting untuk teknik"
]

# Inisialisasi vectorizer dengan parameter default
vectorizer = TfidfVectorizer()

# Fit: membangun vocabulary dan IDF dari korpus
# Transform: menghitung TF-IDF untuk setiap dokumen
tfidf_matrix = vectorizer.fit_transform(dokumen)

# Menampilkan vocabulary (term-term unik)
print("Vocabulary:")
print(vectorizer.get_feature_names_out())

# Menampilkan matriks TF-IDF sebagai array
print("\nMatriks TF-IDF:")
print(tfidf_matrix.toarray())

# Menampilkan nama fitur dan nilainya untuk Dokumen 1
import pandas as pd
df_tfidf = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=vectorizer.get_feature_names_out()
)
print("\nDataFrame TF-IDF:")
print(df_tfidf)
```

Output yang Diharapkan

Vocabulary akan berisi semua term unik setelah tokenisasi. Matriks TF-IDF akan berukuran 2×n (2 dokumen × n term unik). Setiap nilai menunjukkan bobot TF-IDF term tersebut dalam dokumen yang bersangkutan.

Parameter Penting `TfidfVectorizer`

- `max_df` – abaikan term yang muncul di lebih dari x% dokumen
- `min_df` – abaikan term yang muncul di kurang dari n dokumen
- `ngram_range` – gunakan bigram/trigram selain unigram
- `sublinear_tf=True` – aktifkan log normalization untuk TF

Implementasi Manual TF-IDF (Pemahaman Konsep)

Mengimplementasikan TF-IDF secara manual sangat penting untuk membangun intuisi mendalam tentang cara kerja algoritma ini. Berikut adalah langkah-langkah implementasi manual.

1 Hitung TF Setiap Term per Dokumen

Tokenisasi setiap dokumen, lalu hitung frekuensi kemunculan setiap term. Pilih variasi TF yang sesuai: raw count untuk kesederhanaan, atau log normalization untuk hasil yang lebih akurat.

3 Kalikan $TF \times IDF$ untuk Setiap Term

Untuk setiap pasangan (term, dokumen), kalikan nilai TF dengan IDF yang telah dihitung. Hasilnya adalah matriks TF-IDF berukuran (jumlah_dokumen $\times$ ukuran_vocabulary).

2 Hitung Document Frequency (df) dan IDF

Iterasi seluruh koleksi dokumen dan catat berapa banyak dokumen yang mengandung setiap term. Kemudian hitung IDF menggunakan formula: $IDF(t) = \log(N / df(t))$. Tambahkan smoothing jika diperlukan untuk menghindari pembagian nol.

4 Lakukan Normalisasi L2 (Opsional)

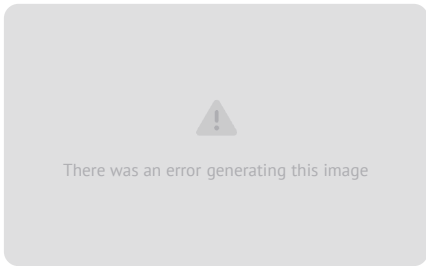
Untuk menggunakan cosine similarity dalam ranking, lakukan normalisasi L2 pada setiap vektor dokumen: bagi setiap nilai TF-IDF dengan norma Euclidean dari vektor tersebut. Ini memastikan semua vektor memiliki panjang 1 sehingga perbandingan cosine menjadi adil terlepas dari panjang dokumen.

BAGIAN 7

Penerapan di Kehidupan Nyata

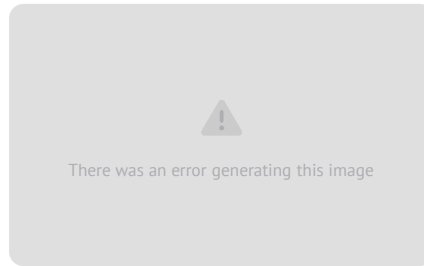
TF-IDF di Platform Digital Populer

TF-IDF dan variasinya digunakan secara luas di platform digital yang kita gunakan sehari-hari. Berikut adalah beberapa contoh penerapan nyata yang paling relevan.



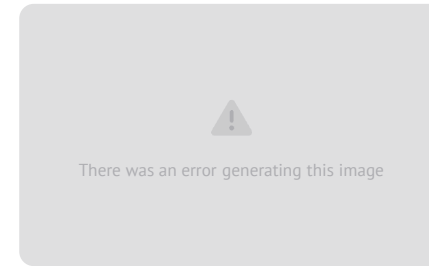
Search Engine (Google & Bing)

Search engine menggunakan TF-IDF (bersama PageRank dan sinyal lainnya) untuk menghitung relevansi halaman web terhadap query pengguna. Halaman dengan TF-IDF tinggi untuk kata kunci pencarian akan muncul di peringkat teratas hasil pencarian.



E-Commerce (Shopee, Tokopedia, Lazada)

Platform belanja online menggunakan TF-IDF pada judul dan deskripsi produk untuk mengurutkan hasil pencarian. Produk yang memiliki bobot TF-IDF tinggi untuk kata kunci yang dicari pengguna akan muncul lebih awal, meningkatkan kemungkinan penjualan.



Rekomendasi Konten (YouTube, TikTok, Spotify)

Platform streaming menganalisis judul, deskripsi, dan tag konten menggunakan TF-IDF untuk memahami topik konten. Hasilnya digunakan untuk merekomendasikan video, lagu, atau film yang relevan dengan preferensi pengguna berdasarkan kesamaan konten.

Penerapan di Bidang AI & Analitik

Deteksi Spam Email

Sistem anti-spam menggunakan TF-IDF untuk mengidentifikasi kata-kata khas spam seperti "gratis", "menang", "klik sekarang". Term-term ini memiliki TF-IDF tinggi dalam email spam dibanding email normal, sehingga mudah dideteksi oleh classifier.

Analisis Sentimen

TF-IDF digunakan sebagai fitur input untuk model klasifikasi sentimen pada ulasan produk, komentar media sosial, atau survei pelanggan. Term dengan bobot tinggi membantu model membedakan ulasan positif, negatif, dan netral.

Repository Akademik

Sistem pencarian di Google Scholar, repository kampus, atau basis data jurnal seperti Scopus menggunakan TF-IDF untuk mencocokkan query penelitian dengan judul, abstrak, dan keyword artikel ilmiah.

Rekomendasi Lowongan Kerja

Platform seperti LinkedIn, Jobstreet, dan Kalibrr menggunakan TF-IDF untuk mencocokkan deskripsi pekerjaan dengan profil dan resume kandidat, menghasilkan rekomendasi lowongan yang relevan.

Personalisasi Berita

Aplikasi berita seperti Google News menggunakan TF-IDF pada konten artikel untuk memahami topik dan mempersonalisasi feed berita sesuai dengan minat pembaca yang dipelajari dari riwayat bacaan.

Studi Kasus: Pencarian Produk E-Commerce

Query pengguna: **"laptop gaming murah"**

Proses di Balik Layar

Ketika pengguna mengetikkan "laptop gaming murah" di kolom pencarian, sistem e-commerce melakukan serangkaian proses TF-IDF secara real-time:

1. Query dipecah menjadi token: ["laptop", "gaming", "murah"]
2. Sistem mengambil vektor TF-IDF yang telah dihitung untuk setiap produk
3. Query juga direpresentasikan sebagai vektor TF-IDF
4. Cosine similarity dihitung antara query vector dan setiap product vector
5. Produk diurutkan berdasarkan skor similarity tertinggi

Mengapa TF-IDF Efektif di Sini?

Kata "laptop" memiliki IDF sedang – muncul di banyak produk elektronik. Kata "gaming" lebih spesifik – IDF lebih tinggi karena tidak semua laptop adalah gaming laptop. Kata "murah" memiliki IDF tertentu karena tidak semua deskripsi produk menyebutkannya.

Produk yang mengandung ketiga kata ini dengan bobot tinggi dalam deskripsinya akan mendapatkan skor cosine similarity tertinggi dan muncul di urutan teratas – itulah kekuatan TF-IDF dalam aplikasi pencarian nyata.

📌 Sistem modern menggabungkan TF-IDF dengan sinyal lain seperti popularitas produk, rating, dan riwayat klik untuk hasil yang lebih optimal.

BAGIAN 8

Latihan & Tugas



Latihan & Tugas Pertemuan Ke-5

Tugas berikut dirancang untuk membangun pemahaman mendalam dari teori ke praktik. Kerjakan secara berurutan untuk memaksimalkan pemahaman konsep.

1

Perhitungan Manual TF-IDF

Buat 3 dokumen pendek (minimal 5 kata per dokumen) dengan topik bebas. Hitung secara manual: (a) Raw TF untuk setiap term, (b) IDF dari ketiga dokumen, (c) TF-IDF akhir. Sajikan hasil dalam tabel yang terstruktur dan interpretasikan hasilnya.

2

Perbandingan Raw TF vs Log TF

Gunakan dokumen yang sama dari Soal 1. Bandingkan nilai TF menggunakan raw count dan log normalization ($1 + \log(f)$). Buat grafik perbandingan dan jelaskan dalam 3-5 kalimat: kapan log normalization memberikan hasil yang lebih baik?

3

Implementasi Python dengan Scikit-learn

Implementasikan TF-IDF menggunakan `TfidfVectorizer` pada corpus minimal 5 dokumen (dapat menggunakan teks berita atau Wikipedia). Tampilkan matriks TF-IDF sebagai DataFrame, identifikasi 3 term dengan bobot tertinggi per dokumen, dan visualisasikan hasilnya menggunakan heatmap.

4

Analisis: TF saja vs TF-IDF

Lakukan pencarian sederhana (ranking dokumen untuk 1 query) menggunakan (a) hanya TF dan (b) TF-IDF. Bandingkan urutan ranking yang dihasilkan. Diskusikan: Apa yang terjadi jika IDF diabaikan? Dalam kondisi apa ranking berbeda signifikan? Apakah ada kondisi di mana TF saja sudah cukup?

Ringkasan Pertemuan Ke-5

Bag-of-Words

Fondasi representasi dokumen: teks diubah menjadi vektor frekuensi kata tanpa memperhatikan urutan. Basis dari semua model IR klasik.

Term Frequency (TF)

Mengukur kepentingan term secara **lokal** di dalam satu dokumen. Variasi: raw, boolean, log, augmented. TF tinggi = term dominan di dokumen.

IDF

Mengukur kepentingan term secara **global** di seluruh koleksi. IDF tinggi = term langka = lebih diskriminatif antar dokumen.

TF-IDF

Perkalian $TF \times IDF$ menghasilkan bobot optimal. Term yang sering di satu dokumen namun jarang di koleksi mendapat skor tertinggi.

Relevansi Saat Ini

Meskipun BERT dan model neural lebih canggih, TF-IDF tetap relevan, cepat, dan efektif – standar baseline yang wajib dikuasai.

Terima Kasih & Diskusi

Pertanyaan Diskusi

Kapan TF-IDF masih lebih baik daripada word embeddings (seperti Word2Vec atau BERT)?

Pertimbangkan aspek-aspek berikut dalam diskusi:

- Ukuran dataset yang tersedia untuk training
- Kebutuhan interpretabilitas vs akurasi
- Keterbatasan komputasi dan latensi sistem
- Jenis query: keyword-based vs semantic search
- Bahasa yang didukung (TF-IDF language-agnostic)

Persiapan Pertemuan Berikutnya

Pertemuan Ke-6: Model Vector Space & Cosine Similarity

Pastikan Anda memahami dengan baik:

- Representasi dokumen sebagai vektor (BoW & TF-IDF)
- Operasi dot product antara dua vektor
- Konsep dasar norma vektor (Euclidean norm)

Referensi Utama

Manning, C.D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*, Chapter 6: Scoring, Term Weighting and the Vector Space Model. Cambridge University Press.

Tersedia gratis online: nlp.stanford.edu/IR-book

